



códigofacilito



Futuros y async

Bootcamp Python Avanzado.

Eduardo I. García Pérez.





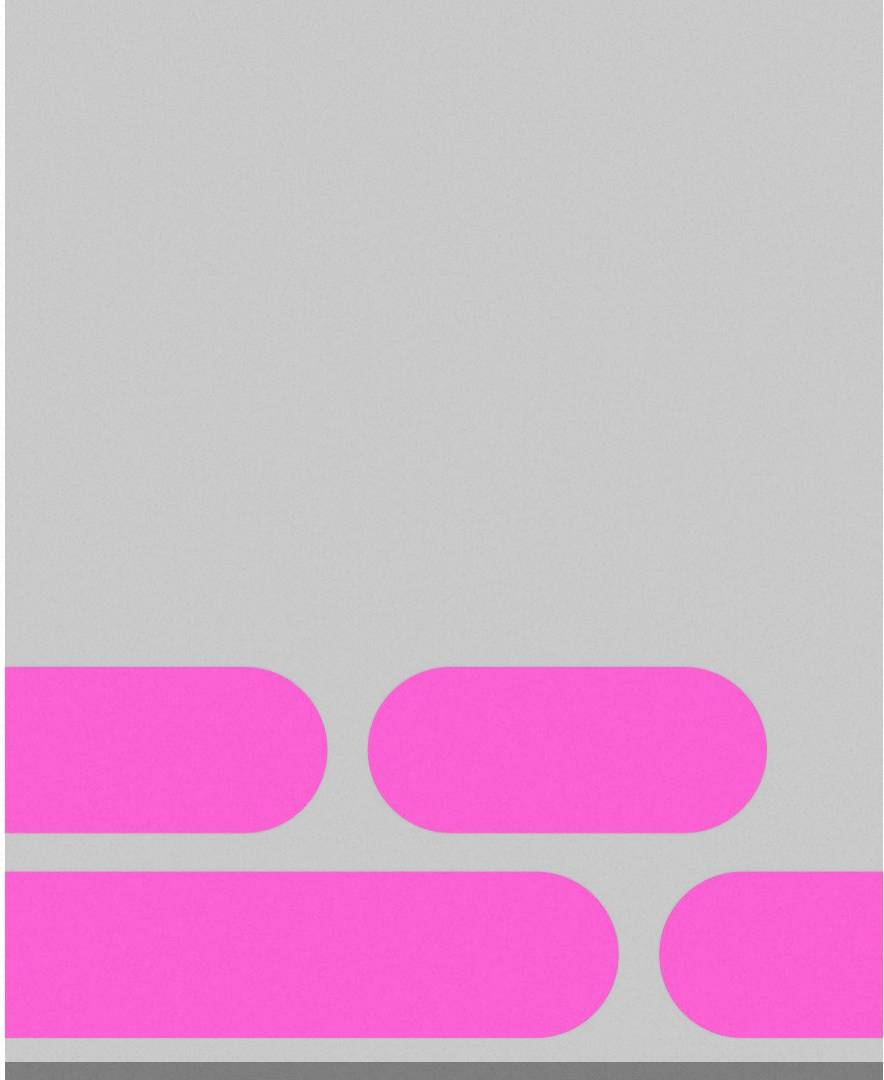
>_ Temario.

- Repaso de Threads y procesos
- Pool de threads y procesos.
- Futuros
- Coroutines
- Funciones syncs.





Repaso General





>_ ¿Qué son los procesos? 🌱

- Un proceso es una instancia en ejecución de un programa.
- Tiene su propio espacio de memoria, conjunto de recursos y estado.
- Los procesos se crean y se administran por el sistema operativo.





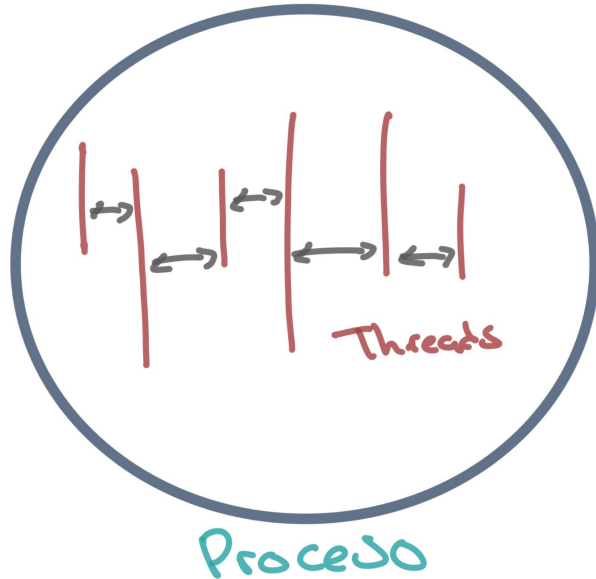
>_ ¿Qué son los Threads? 🐢

- Un thread es una secuencia de instrucciones que se ejecuta dentro de un proceso.
- Un proceso puede tener múltiples hilos que comparten el mismo espacio de memoria y recursos, lo que permite una comunicación y sincronización más fácil entre ellos.
- Los hilos se ejecutan de forma concurrente.



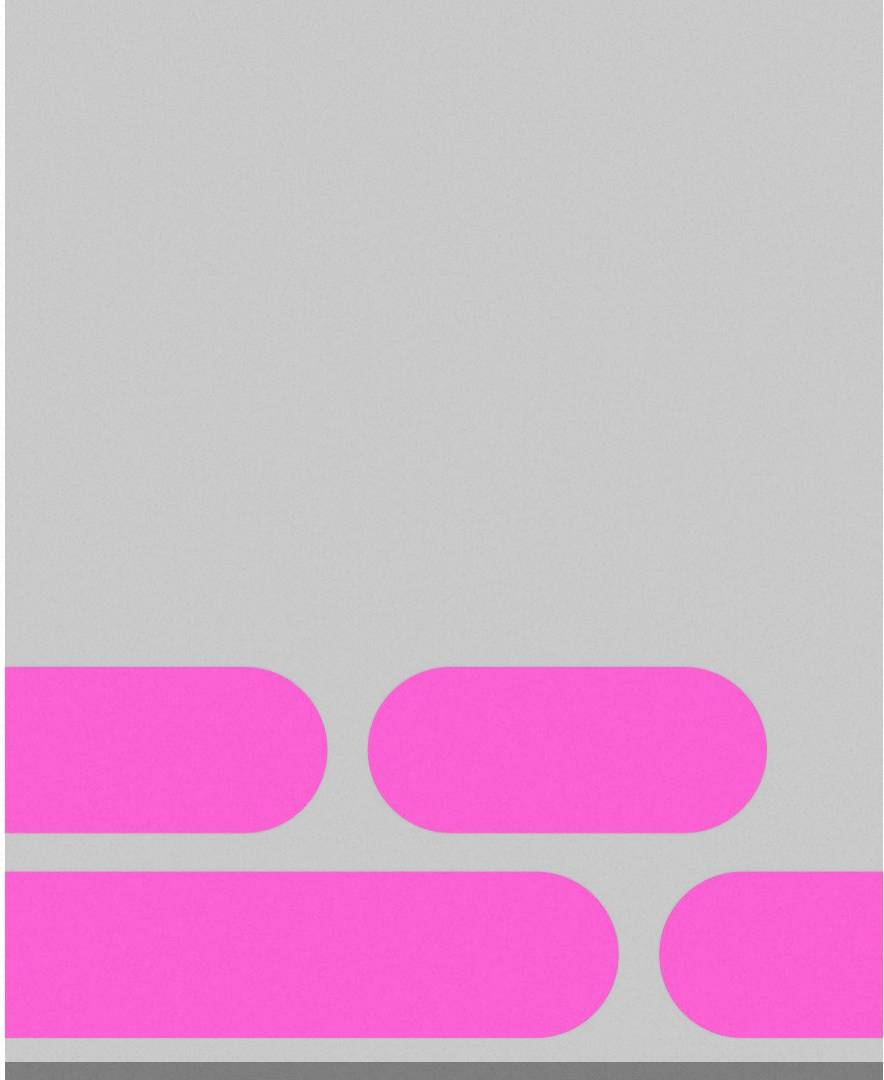


Threads (Concurrencia) y procesos (Paralelismo) 🐢





Pool de threads y procesos.





>_ ¿Qué es una pool?

- Es una estructura que permite gestionar la creación y la ejecución de threads y/o procesos.
- Permite crear una cantidad **Limitada** de Threads y/o procesos.
- Permite re-utilizar Threads y/o procesos que se encuentren disponibles, pudiendo así mejorar el performance de la aplicación.





>_ Creación de una pool

```
>>> from concurrent.futures import ThreadPoolExecutor
```

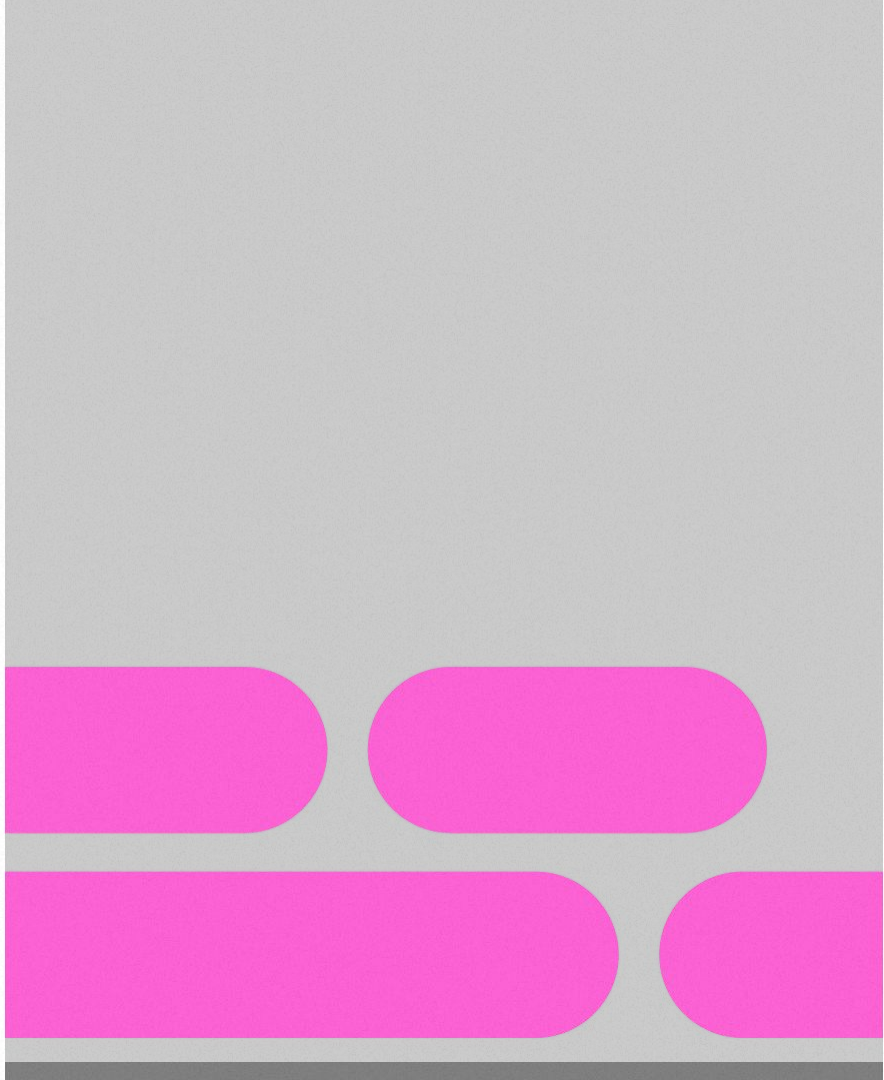
```
>>> with ThreadPoolExecutor() as executor:
```

```
>>>     pass
```





Futuros





>_ ¿Qué es un futuro? 🦎

- Es una abstracción que permite llevar la gestión de un resultado asíncrono. Algo que aún no sucede, pero, **posiblemente**, suceda.





>_ ¿Cómo se crean los futuros? 🐢

```
>>> from concurrent.futures import Future
```

```
>>> future = Future()
```

```
>>> future.set(True)
```

```
>>> future.set_result(<Value>)
```





>_ Métodos de futuros

- **result** = Retorna el resultado asíncrono.
- **set_result** = Define el resultado asíncrono.
- **set_exception** = Marca el futuro como Done y lanza una excepción.
- **done** = Retorna True si el futuro ha finalizado.
- **cancelled** = Retorna True si el futuro ha sido cancelado.
- **add_done_callback** = Define un callback a ejecutarse.
- **cancel** = Cancela el futuro.
- **exception** = Retorna la excepción programada.





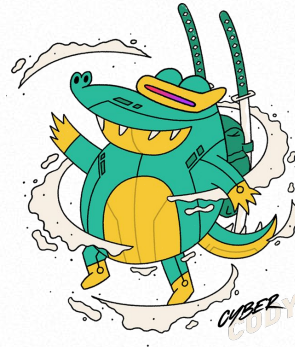
Futuros en pool de Threads y procesos.



Es posible implementar futuros en futuros.

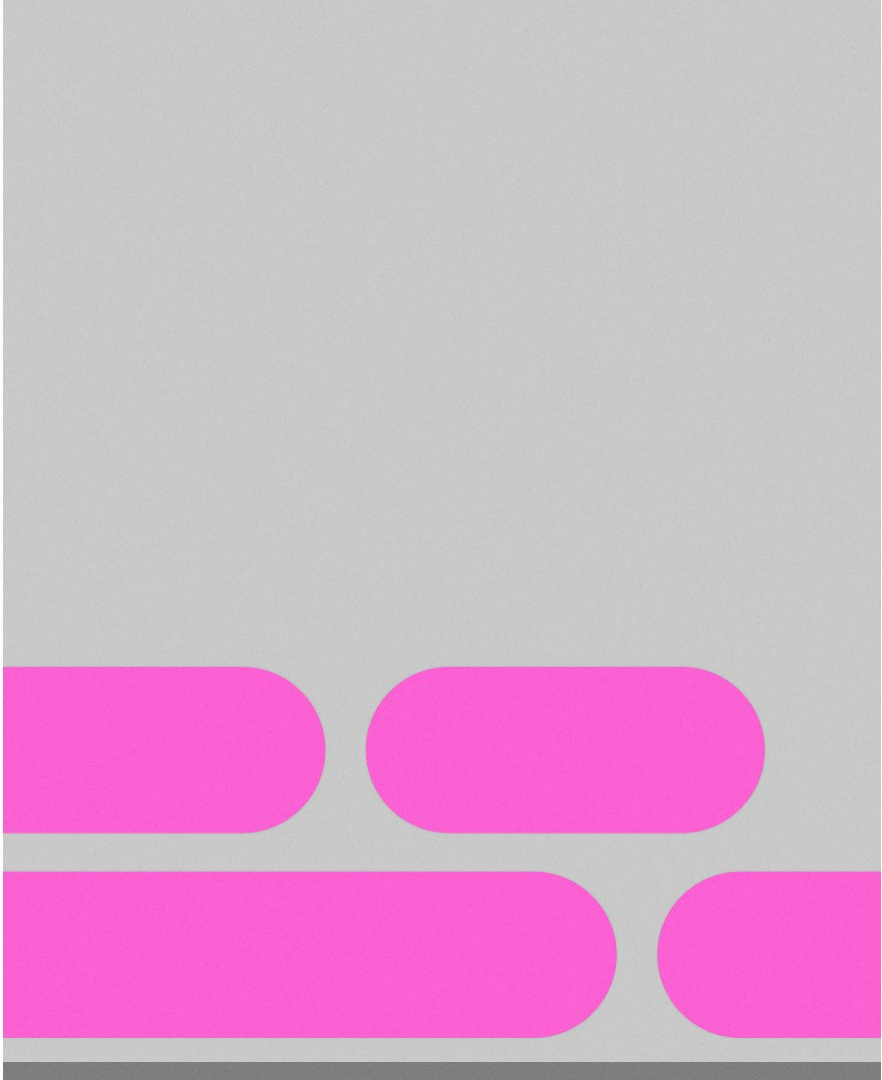
>>> ThreadPoolExecutor

>>> ProcessPoolExecutor





Coroutines





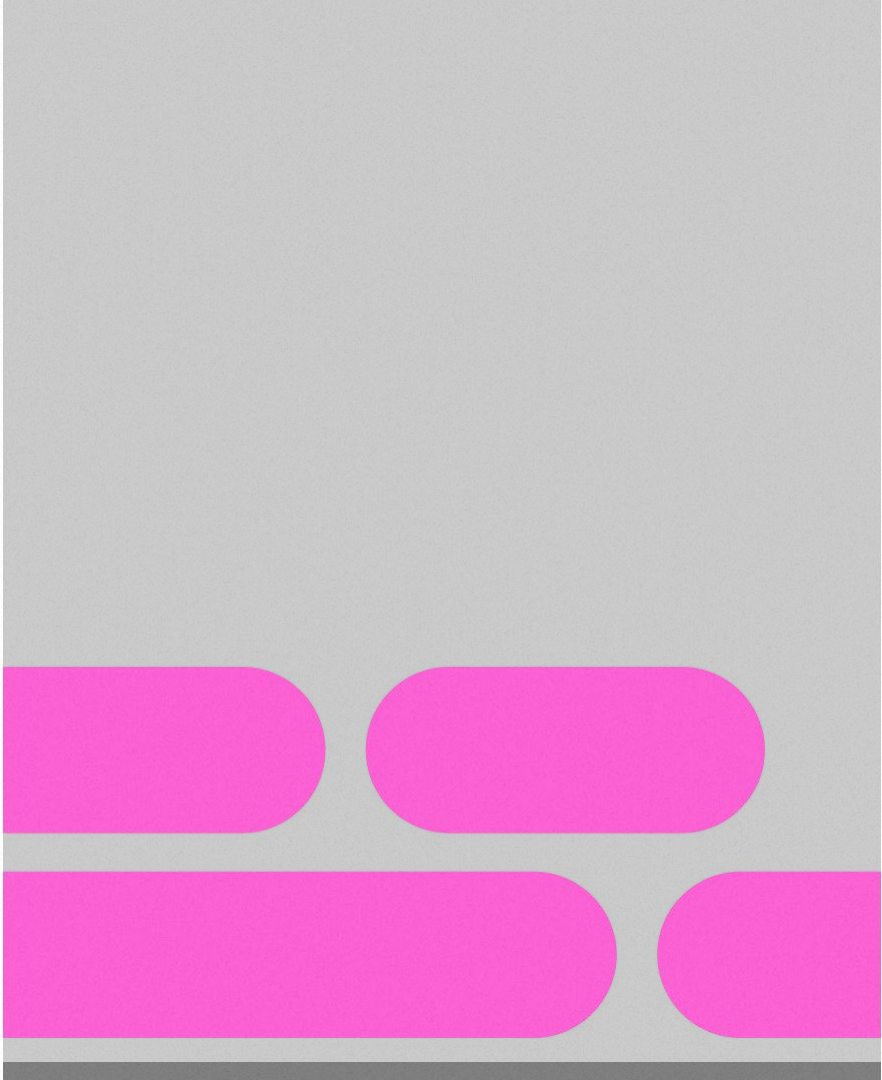
>_ ¿Qué son las Coroutines?

- Es una función que pueda pausar su ejecución y reanudarla en otro momento.
- La función se reanudará cuando la tarea haya sido completada.





Funciones asíncronas.





>_ ¿Cómo se crean las funciones async?

- Es necesario importar el módulo `asyncio`.
- Es necesario hacer uso de las palabras reservadas **`async`** & **`await`**.

```
>>> asyncio.run()  
>>> asyncio.create_task()
```





>_ ¿Threads, Procesos o Async?

- La corutinas permite ejecutar tareas de manera concurrente dentro de un único hilo/proceso.
- Los hilos permiten la ejecución concurrente de varias tareas dentro de un único proceso.
- Los procesos son unidades de ejecución completamente independientes. Se aprovechan todos los recursos de la computadora.





códigofacilito



Futuros y async

Bootcamp Python Avanzado.

Eduardo I. García Pérez.

